

App Dev Project Report

1. Student Details

- Name: Satyanshi Singh
- Roll Number: 23f3004132
- Email: 23f3004132@ds.study.iitm.ac.in

2. Project Details

Project Title

Placement Portal Application (MAD-2)

Problem Statement

Build a role-based web application for campus placement management where:

- Admin can manage students, companies, and placement drives.
- Companies can create recruitment drives, manage applications, and update candidate status.
- Students can search eligible drives, upload resumes, apply for jobs, and track application status.
- System supports reminders, reports, search functionality, caching, and CSV export.

My Approach

I built the project as a frontend-backend application:

- **Backend:** Flask REST APIs with SQLAlchemy models and role-based authentication.
- **Frontend:** Vue.js dashboards for Admin, Company, and Student.
- **Async Jobs:** Celery + Redis for scheduled emails and CSV export tasks.
- **Data Storage:** SQLite database.

I followed a modular structure with separate layers for:

- Resources (API route handlers)
- Services (business logic)
- Models (database schema)
- Tasks (background jobs)

AI/LLM Declaration

I used AI/LLM tools (ChatGPT/GitHub Copilot) for limited support during development.

Extent of Use

Approximate usage: 20–25%

Used for:

- API/resource boilerplate
- Error fixing
- Celery & Redis setup
- Vue components
- CSS styling

Not used for:

- Final design decisions
- Core business logic
- End-to-end implementation

All integration, testing, debugging, and project decisions were completed manually.

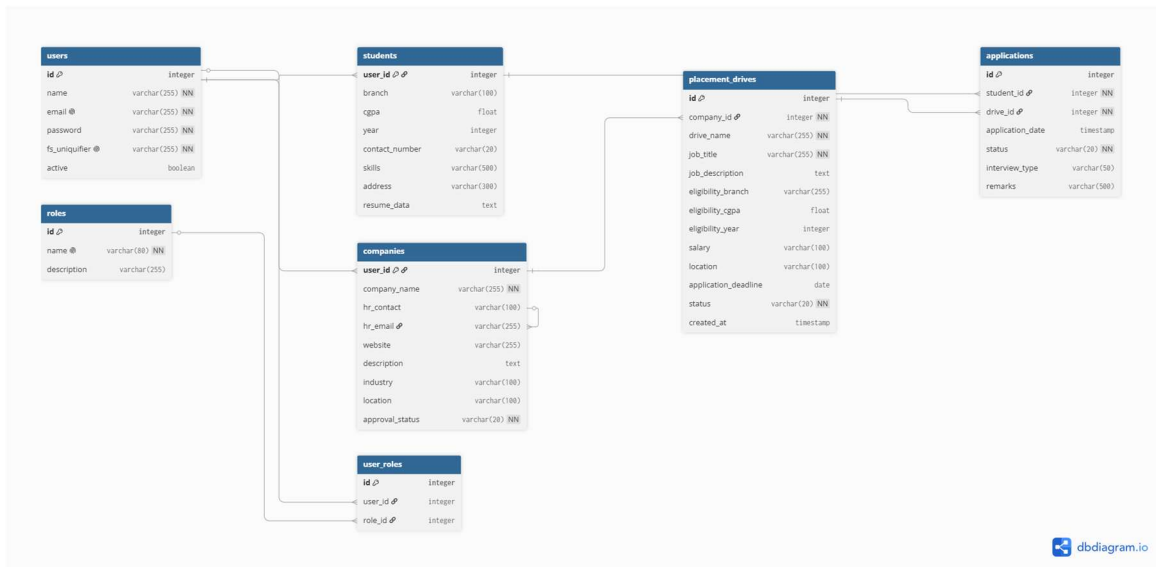
4. Frameworks and Libraries Used

Technology / Library	Purpose in Project
Flask	Backend
Flask-RESTful	API routing
Flask-Security-Too	Authentication
SQLAlchemy	ORM
SQLite	Database
Vue.js	Frontend
Vue Router	Routing
Vuex	State Management
Fetch API	Communication
Celery	Background Jobs
Redis	Caching & Broker
Flask-Caching	Caching
Bootstrap 5	UI
Jinja2	Email Templates

5. Database ER Diagram

Main Relationships

- users ↔ roles
- users → students
- users → companies
- companies → drives
- students → applications
- drives → applications



6. API Resource Endpoints

Auth

- POST [/api/login](#)
- POST [/api/register/student](#)
- POST [/api/register/company](#)

Students

- GET [/api/students](#)
- GET [/api/students/<id>](#)
- PATCH [/api/students/<id>](#)
- DELETE [/api/students/<id>](#)
- POST [/api/students/<id>/resume](#)
- GET [/api/students/<id>/resume](#)
- POST [/api/students/<id>/export-csv](#)
- GET [/api/students/<id>/export-csv/<task_id>](#)
- GET [/api/students/<id>/export-csv/<task_id>/download](#)
- GET [/api/students/search](#)

Companies

- GET [/api/companies](#)
- GET [/api/companies/<id>](#)
- PATCH [/api/companies/<id>](#)
- DELETE [/api/companies/<id>](#)
- GET [/api/companies/search](#)

Drives

- GET [/api/drives](#)
- POST [/api/drives](#)
- GET [/api/drives/<id>](#)
- PATCH [/api/drives/<id>](#)
- DELETE [/api/drives/<id>](#)
- GET [/api/drives/search](#)

Applications

- GET [/api/applications](#)
- POST [/api/applications](#)
- GET [/api/applications/<id>](#)
- PATCH [/api/applications/<id>](#)
- DELETE [/api/applications/<id>](#)

7. Key Features Implemented

- Role-based dashboards
- Company approval workflow
- Drive approval workflow
- Resume upload
- Eligibility validation

- Duplicate application prevention
- Application status tracking
- Search
- CSV export
- Daily reminder emails
- Monthly reports
- Redis caching

8. Video Presentation

Video Link –

<https://drive.google.com/file/d/12aMQ1JtawMBgoiX5FDznguSRmQyt4Beq/view?usp=sharing>

9. Conclusion

This project helped me understand full-stack application development using Flask and Vue.js, REST APIs, database modeling, asynchronous task processing, and caching. It improved my understanding of modular architecture and real-world campus placement workflows.